Feature Engineering

Mohon membaca setiap instruksi dan tag komentar dengan saksama sebelum melanjutkan pengerjaan.

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split

# Scalling untuk MinMax Scaller
from sklearn.preprocessing import MinMaxScaler
# Set the maximum number of columns and rows to display to a large number
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
```

B. Outlier Handling

Outlier Handling : InterQuartile Handling

In [4]:
```
df_california = pd.read_csv('california_dataset.csv')
```

In [2]:
```
# untuk plot Q-Q
import scipy.stats as stats
```

In [5]:
```
df_california.head()
```

Out[5]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

In [5]:
```
from sklearn.model_selection import train_test_split
# splitting terlebih dahulu


#NOTES :
#train : test = 80:20 atau 75:25 atau (minimum :70:30 atau maksimum: 90:10) bagi an
```

```
#data train di-handling outliernya
#tapi tidak untuk data test, karena data test ibarat representasi data masa depan y
```

In [6]:
```
df_california.columns.tolist()
```

Out[6]:
```
['MedInc',
 'HouseAge',
 'AveRooms',
 'AveBedrms',
 'Population',
 'AveOccup',
 'Latitude',
 'Longitude',
 'house_price']
```

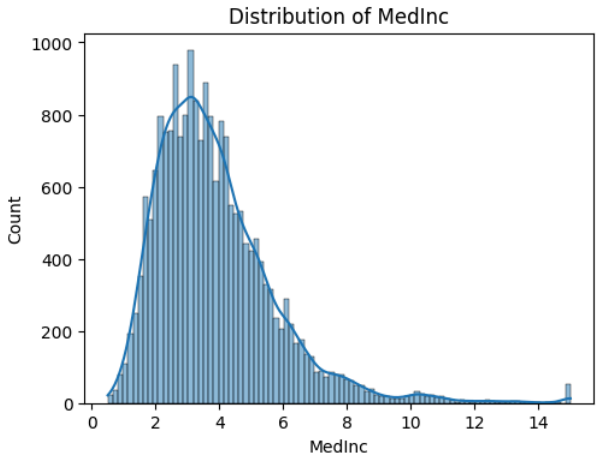
Assignment 1.a :

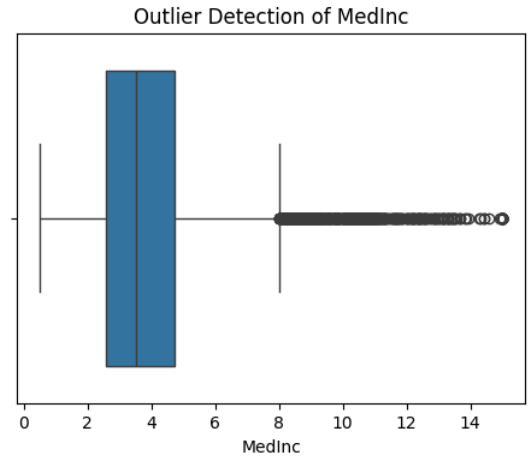
1. lakukan looping untuk melihat plot distribusi data dan outlier pada kolom-kolom : ['MedInc', 'HouseAge', 'AveRooms','AveBedrms', 'AveOccup'] di california_dataset.csv
2. Daftar nama kolom yang Anda cek, kolom_kolom = ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'AveOccup']

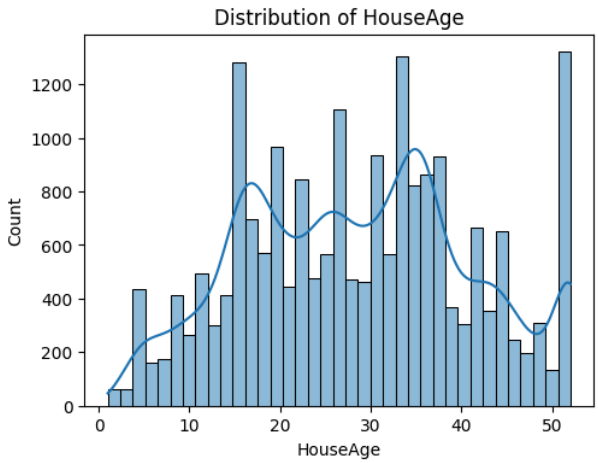
In [7]:
```
df_california.head()
df_california.columns
```

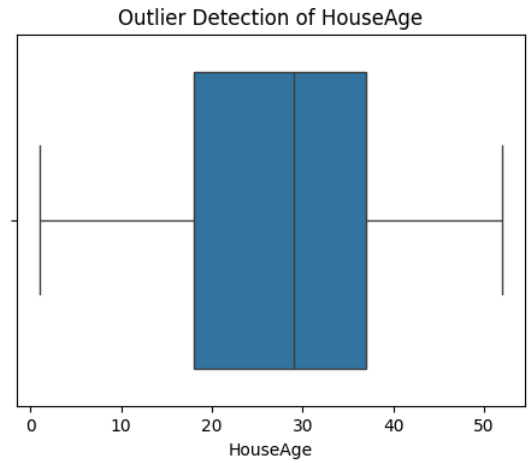
Out[7]:
```
Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
       'Latitude', 'Longitude', 'house_price'],
      dtype='object')
```

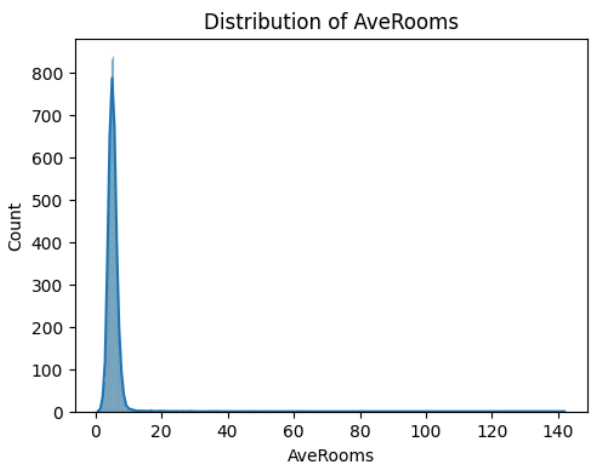
In [8]:
```
columns = df_california.columns

for col in columns:

    if df_california[col].dtype != 'object':

        plt.figure(figsize=(12,4))

        plt.subplot(1,2,1)
        sns.histplot(df_california[col], kde=True)
        plt.title(f'Distribution of {col}')

        plt.subplot(1,2,2)
        sns.boxplot(x=df_california[col])
        plt.title(f'Outlier Detection of {col}')

        plt.show()
```

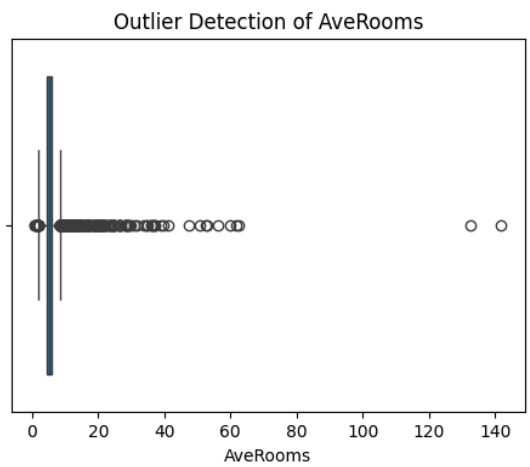

Distribution of MedInc
Count
MedInc

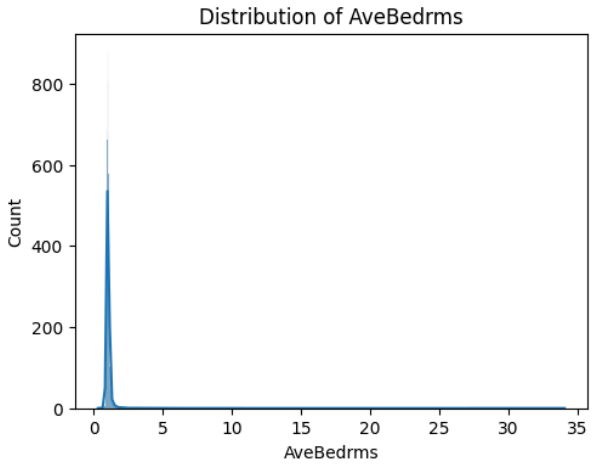

Outlier Detection of MedInc
MedInc

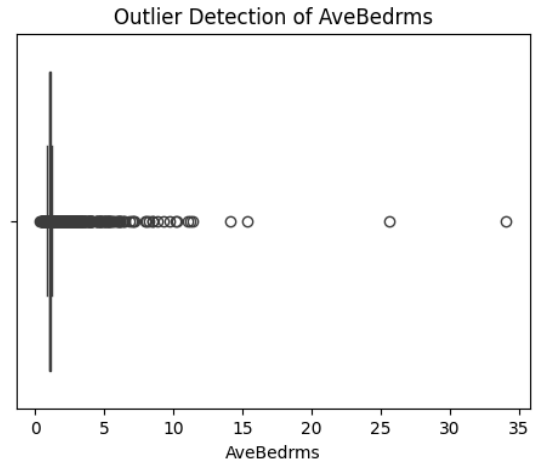

Distribution of HouseAge
Count
HouseAge

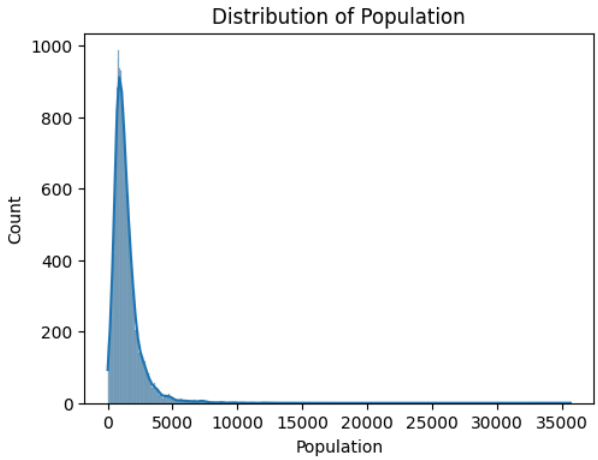

Outlier Detection of HouseAge
HouseAge

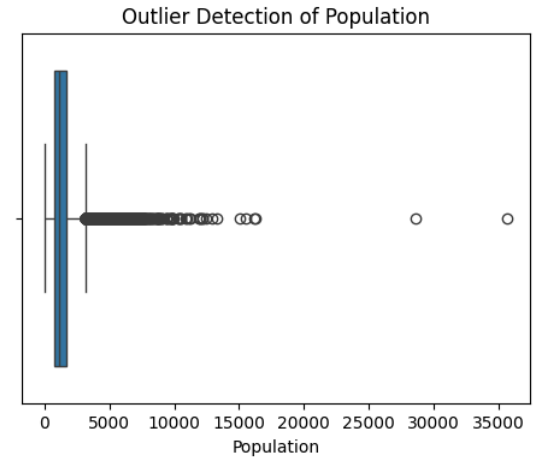

Distribution of AveRooms
Count
AveRooms

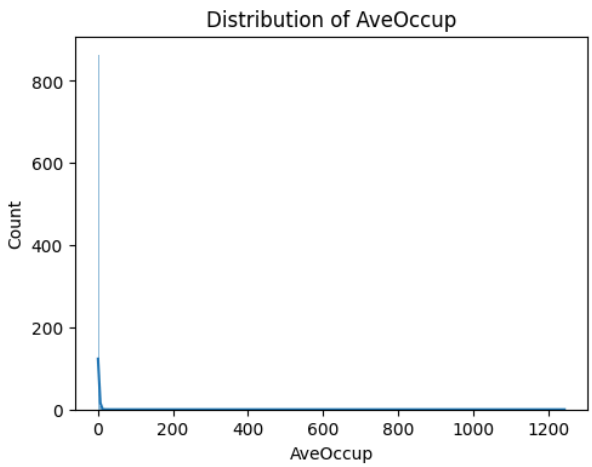

Outlier Detection of AveRooms
AveRooms

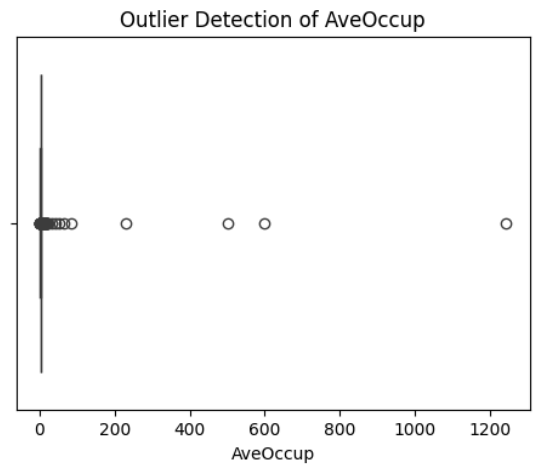

Distribution of AveBedrms
Count
AveBedrms

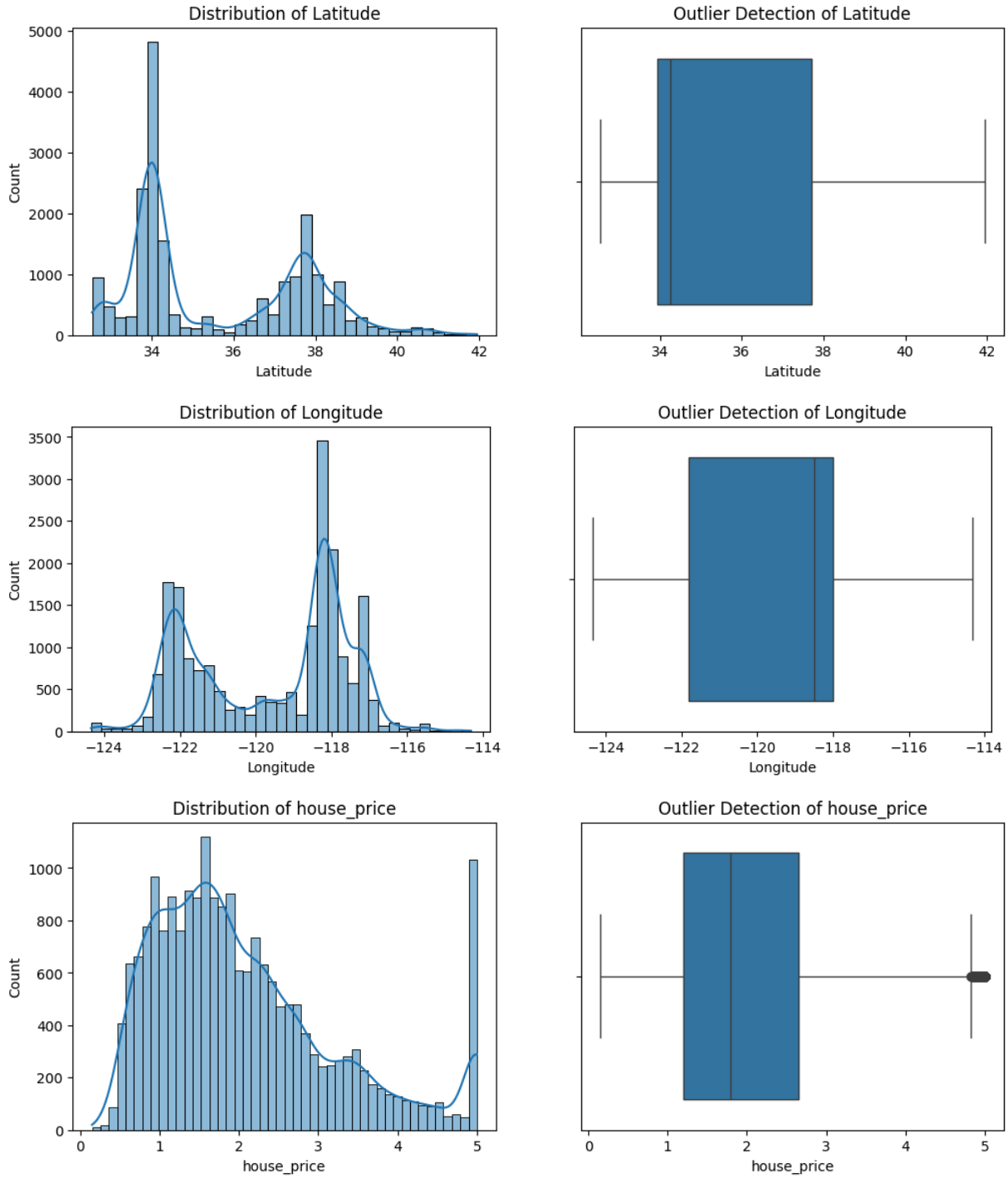

Outlier Detection of AveBedrms
AveBedrms


Distribution of Population
Count
Population


Outlier Detection of Population
Population


Distribution of AveOccup
Count
AveOccup


Outlier Detection of AveOccup
AveOccup



In [9]:
```
df_california = df_california.rename(columns={'median_income': 'MedInc', 'housing_m
```

In [10]:
```
display(df_california.head())
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

Assignment 1.b

1. Lakukan handling outlier pilih 1 kolom saja yang di handling dengan IQR setelah mengecek distribusinya
2. Dan cek distribusinya kembali

In [17]:
```
col = 'AveRooms'
```

In [18]:
```
Q1 = df_california[col].quantile(0.25)
Q3 = df_california[col].quantile(0.75)

print("Q1 :", Q1)
print("Q3 :", Q3)
```

```
Q1 : 4.440716235896959
Q3 : 6.052380952380952
```

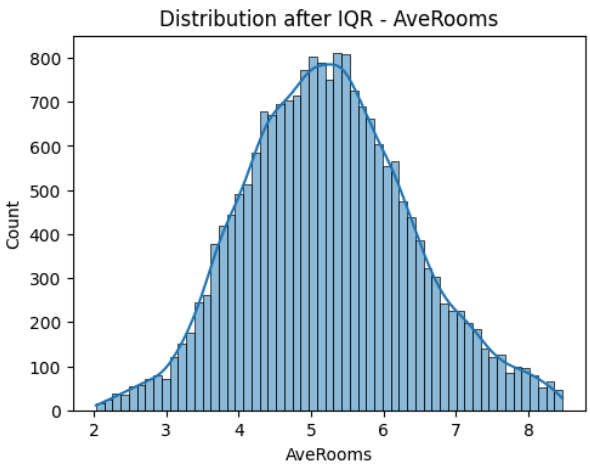
In [19]:
```
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
```

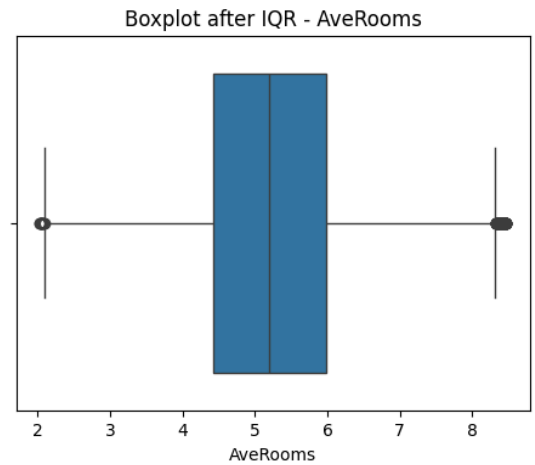
```
Lower Bound: 2.023219161170969
Upper Bound: 8.469878027106942
```

In [20]:
```
df_no_outlier = df_california[(df_california[col] >= lower_bound) & (df_california[
```

In [21]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
sns.histplot(df_no_outlier[col], kde=True)
plt.title(f'Distribution after IQR - {col}')

plt.subplot(1,2,2)
```

```
sns.boxplot(x=df_no_outlier[col])
plt.title(f'Boxplot after IQR - {col}')

plt.show()
```





C. Missing Value Handling

Missing Value Merupakan nilai yang hilang di suatu kolom, namun tidak hanya sebatas itu. Untuk nilai yang tidak sesuai tapi muncul didalam suatu kolom tertentu juga bisa dianggap sebagai missing value. Misal tipenya kategorikal tapi isinya ada yang berupa angka dan juga sebaliknya.

In [23]:
```
import pandas as pd

df_company = pd.read_csv('company.csv')

df_company.head()
```

Out[23]:

	Revenue	Size	Rating	Headquarters
0	Unknown / Non-Applicable	1001 to 5000 employees	3.1	New York, NY
1	$1 to 2$ billion (USD)	5001 to 10000 employees	4.2	Herndon, VA
2	$100 to 500$ million (USD)	1001 to 5000 employees	3.8	Boston, MA
3	$100 to 500$ million (USD)	501 to 1000 employees	3.5	Bad Ragaz, Switzerland
4	Unknown / Non-Applicable	51 to 200 employees	2.9	New York, NY

Aturan missing value handling :

1. jika tipe datanya itu **numerik** -> handling menggunakan **median** (robust / tahan terhadap outlier)
2. jika tipe datanya itu **kategorik / object / string** -> handling menggunakan **mode / modus**

In [24]:
```
df_company.isnull().sum()
```

Out[24]:

	0
Revenue	0
Size	0
Rating	0
Headquarters	0

dtype: int64

In [25]:
```
num_cols = df_company.select_dtypes(include=['int64','float64']).columns
cat_cols = df_company.select_dtypes(include=['object']).columns

print("Numerical Columns:", num_cols)
print("Categorical Columns:", cat_cols)
```

```
Numerical Columns: Index(['Rating'], dtype='object')
Categorical Columns: Index(['Revenue', 'Size', 'Headquarters'], dtype='object')
```

In [26]:
```
# Handling numerik dengan median
for col in num_cols:
    df_company[col].fillna(df_company[col].median(), inplace=True)

# Handling kategorik dengan mode
for col in cat_cols:
    df_company[col].fillna(df_company[col].mode()[0], inplace=True)
```

```
/tmp/ipykernel_376/2727344298.py:3: FutureWarning: A value is trying to be set on a
copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
the operation inplace on the original object.


  df_company[col].fillna(df_company[col].median(), inplace=True)
/tmp/ipykernel_376/2727344298.py:7: FutureWarning: A value is trying to be set on a
copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
the operation inplace on the original object.


  df_company[col].fillna(df_company[col].mode()[0], inplace=True)
```

In [27]:
```
df_company.isnull().sum()
```

Out[27]:

	0
Revenue	0
Size	0
Rating	0
Headquarters	0

dtype: int64

In [30]:
```
from sklearn.model_selection import train_test_split

# Tentukan target
X = df_company.drop('Revenue', axis=1)
y = df_company['Revenue']

# Split data 80:20
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (537, 3)
Test shape: (135, 3)
```

Assignment 2. Missing Value Handling :

1. Cek persentase missing value pada kolom Headquarters di company.csv, Cek jenis missing valuenya ?
2. apakah di drop atau tidak (gunakan syarat drop kolom jika > 20%)
3. jika tidak maka handling, dan dengan cara apa handling nilainya, median atau modus ?

In [31]:
```
#Tulis jawaban missing value handling kolom Headquarters disini
# cek jumlah missing value
missing = df_company['Headquarters'].isnull().sum()

# total data
total = len(df_company)

# persentase
percentage = (missing / total) * 100

print("Jumlah missing value :", missing)
```

```
print("Total data :", total)
print("Persentase missing value :", percentage, "%")
```

```
Jumlah missing value : 0
Total data : 672
Persentase missing value : 0.0 %
```

In [32]:
```
df_company['Headquarters'].dtype
```

Out[32]:
```
dtype('O')
```

In [35]:
```
# cek missing value
missing = df_company['Headquarters'].isnull().sum()
total = len(df_company)
percentage = (missing / total) * 100

print("Missing value:", missing)
print("Persentase:", percentage, "%")

# jika <= 20% lakukan handling dengan modus
if percentage <= 20:
    df_company['Headquarters'] = df_company['Headquarters'].fillna(df_company['Head
    print("Missing value diisi menggunakan modus")
else:
    df_company = df_company.drop(columns=['Headquarters'])
    print("Kolom di-drop karena missing value > 20%")
```

```
Missing value: 0
Persentase: 0.0 %
Missing value diisi menggunakan modus
```

In [36]:
```
df_company['Headquarters'] = df_company['Headquarters'].fillna(...)
```

D. Encoding

Definisi : Mengubah tipe kategorikal menjadi numerikal dengan pendekatan aturan encoding untuk persiapan sebelum dilakukannya machine learning modelling. Teknik Encoding yang umum dipakai :

1. One Hot Encoding (OHE) : Lakukan pada kolom gender
2. Label Encoding / Ordinal Encoding : Lakukan pada kolom Yes dan No, Jika Yes menjadi 1 , Jika No menjadi 0. Sebagai Contohnya di kolom Partner, Dependents.
3. Mean Encoding : Contract

In [40]:
```
import os
os.listdir()
```

Out[40]:
```
['.config',
 'TelcoCustomerChurn.csv',
 'README.md',
 'company.csv',
 'Hands On_Week_3_Feature_Engineering.ipynb',
 'TelcoCustomerChurn.xlsx',
 'Function_in_Python.json',
 'california_dataset.csv',
 'Assignment_Week 3.ipynb',
 'sample_data']
```

In [41]:
```
df_churn = pd.read_csv('TelcoCustomerChurn.csv')
```

In [42]:
```
df_churn.head()
```

Out[42]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multipl
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

Assignment 3.a Label Encoder :

1. Pilih kolom-kolom StreamingMovies, StreamingTV, TechSupport, DeviceProtection, OnlineBackup, OnlineSecurity, MultipleLines
2. Ganti nilai yang tidak sesuai : No internet service menjadi No lalu validasi dengan value_counts
3. Kemudian lakukan label Encoder
4. Notes : gunakan Telco-Customer-Churn.csv

In [44]:
```
columns = [
'StreamingMovies',
'StreamingTV',
```

```
'TechSupport',
'DeviceProtection',
'OnlineBackup',
'OnlineSecurity',
'MultipleLines'
]
```

In [45]:
```
for col in columns:
    df_churn[col] = df_churn[col].replace('No internet service', 'No')
```

In [46]:
```
for col in columns:
    print(col)
    print(df_churn[col].value_counts())
    print()
```

```
StreamingMovies
StreamingMovies
No     4311
Yes    2732
Name: count, dtype: int64

StreamingTV
StreamingTV
No     4336
Yes    2707
Name: count, dtype: int64

TechSupport
TechSupport
No     4999
Yes    2044
Name: count, dtype: int64

DeviceProtection
DeviceProtection
No     4621
Yes    2422
Name: count, dtype: int64

OnlineBackup
OnlineBackup
No     4614
Yes    2429
Name: count, dtype: int64

OnlineSecurity
OnlineSecurity
No     5024
Yes    2019
Name: count, dtype: int64

MultipleLines
MultipleLines
No                  3390
Yes                 2971
No phone service     682
Name: count, dtype: int64
```

In [47]:
```
df_churn[columns].head()
```

Out[47]:

	StreamingMovies	StreamingTV	TechSupport	DeviceProtection	OnlineBackup	OnlineSe
0	No	No	No	No	Yes	
1	No	No	No	Yes	No	
2	No	No	No	No	Yes	
3	No	No	Yes	Yes	No	
4	No	No	No	No	No	

In [48]:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in columns:
    df_churn[col] = le.fit_transform(df_churn[col])
```

Kolom yang digunakan memiliki nilai kategorikal berupa Yes dan No. Untuk mempersiapkan data sebelum digunakan dalam model machine learning, nilai tersebut diubah menjadi numerik menggunakan metode Label Encoding dimana Yes direpresentasikan sebagai 1 dan No sebagai 0.

Selesai :)

Kesimpulan:

Proses preprocessing data dilakukan melalui beberapa tahap, yaitu eksplorasi distribusi data untuk mendeteksi outlier, penanganan outlier menggunakan metode IQR, penanganan missing value pada kolom Headquarters dengan modus, serta transformasi data kategorikal menjadi numerik menggunakan Label Encoding. Tahapan ini dilakukan untuk mempersiapkan dataset agar dapat digunakan secara optimal dalam proses pemodelan machine learning.